

Multi-Agent AI Co-worker Engine

Source Code Architecture Report

This report explains the mission, simulation context, source-code design, prototype architecture, API surface, retrieval layer, memory model, interaction modes, safety controls, evaluation approach, and limitations of the submitted local FastAPI prototype.

Executive summary: The prototype demonstrates a configurable multi-agent AI co-worker engine with a browser UI, FastAPI API layer, agent personas, RAG-style retrieval, blackboard memory, safety checks, and deterministic mock LLM output for review without paid keys.

1. Problem & mission

The problem is to help a learner complete a Gucci HRM leadership-development simulation without replacing the learner's own reasoning. The system must behave like an AI co-worker: useful, structured, domain-aware, and bounded.

The mission of the source code is to provide a local multi-agent prototype that can route questions, consult specialist personas, retrieve relevant simulation context, maintain state, and return explainable metadata for review.

- Support the learner with scaffolding instead of final-answer outsourcing.
- Expose a local API-first prototype suitable for inspection and testing.
- Keep the system runnable without a paid LLM key through MockLLMClient.

2. Gucci simulation context

The case context is a Group-level HRM and leadership-development assignment for Gucci. The prototype models workstreams around Group DNA, leadership competencies, 360 feedback, coaching, talent mobility, and regional rollout.

Knowledge files in data/ and shared/ provide the assignment context, deliverables, guardrails, tool catalog, and shared prompt template. Agent-specific folders add persona-level behavior, tasks, skills, memory schema, examples, and handoff rules.

- Core themes: Vision, Entrepreneurship, Passion, and Trust.
- Key tension: Group-wide consistency versus brand or regional adaptation.
- Expected learner outputs: competency model, feedback plan, coaching program, rollout plan, and measurement plan.

3. Multi-agent AI co-worker design

The design uses multiple specialized co-workers rather than a single assistant. Each agent has a bounded role and can be selected directly from the UI. The boss agent can coordinate the team when the user needs synthesis across domains.

The code separates persona configuration from runtime orchestration. This makes the prototype easier to extend with new agents or new workflows without rewriting the API layer.

- Config-driven agents live under agents/<agent_id>/.
- Runtime behavior is handled by BaseAgent, MultiAgentOrchestrator, IntentRouter, PromptBuilder, and LLM clients.
- The UI deliberately hides system prompts while allowing agent switching.

4. Agent personas: CEO, CHRO, Regional Manager

The CEO persona focuses on enterprise strategy, Group DNA, governance, and tradeoffs between Group non-negotiables and local autonomy. The CHRO persona focuses on competencies, 360 feedback, coaching, mobility, succession, and development integrity.

The Regional Manager persona focuses on rollout, localization, communication channels, trainer readiness, employee voice, and adoption metrics. Together, these personas cover strategy, people-system design, and implementation.

- CEO: pressure-tests strategic clarity and executive tradeoffs.
- CHRO: makes HR design observable, fair, developmental, and measurable.
- Regional Manager: turns plans into localized adoption and communication workflows.

5. Multi-agent architecture

The central runtime object is MultiAgentOrchestrator. It constructs loader, retriever, prompt builder, LLM client, intent router, message bus, blackboard, memory manager, supervisor, synthesizer, safety checker, and tool router.

The architecture supports direct response, consultation, panel responses, debate synthesis, and handoff behavior. Supporting agents communicate through the MessageBus, while final user-facing responses are produced by the active primary agent or the synthesizer.

- AgentLoader loads YAML and Markdown persona modules.
- IntentRouter decides primary and supporting agents.
- Synthesizer merges panel or debate outputs.
- MessageBus records private inter-agent consultation messages.

6. API layer

The FastAPI layer is intentionally small. app.py creates the FastAPI app, mounts the static UI, creates the default orchestrator, and includes routes from src/api/routes.py.

The API supports /health, /agents, /api/agents, /chat, /chat/{agent_id}, and /api/chat compatibility routes. Pydantic schemas in src/api/schemas.py define request and response shapes.

- GET /health returns service status.
- GET /api/agents returns the default agent and available agents.
- POST /chat supports advanced mode selection.
- POST /chat/{agent_id} supports the browser UI's selected-agent workflow.

7. RAG + Vector DB design

The retrieval layer builds chunks from data/, shared/, and agents/ files. MockEmbeddingClient creates deterministic local embeddings, while MockVectorDB provides an in-memory vector-store interface with cosine similarity.

The design intentionally mimics production RAG shape without requiring FAISS or hosted embeddings. It also filters retrieved context by agent scope where possible and falls back to broader retrieval when needed.

- RAGIndexBuilder chunks Markdown and YAML knowledge files.
- Retriever.retrieve returns scored raw chunks for internal prompt construction.
-

Public API responses expose metadata only, not full retrieved prompt text.

- `system_prompt.md` files are excluded from the public retrieval index.

8. Memory + shared blackboard

The prototype uses two memory concepts. `MemoryStore` keeps per-session conversational state for the agent runtime, while `Blackboard` stores shared project state such as current module, active deliverable, missing outputs, decisions, risks, and retrieval history.

The blackboard lets future turns inherit context from previous turns without exposing hidden prompt logic. It also helps the orchestrator track deliverable progress and debugging metadata.

- `Blackboard.load` initializes session state.
- `Blackboard.update_from_turn` records module, intent, last response summary, and retrieval history.
- `MemoryStore` tracks session-level confidence, completed deliverables, and conversation history.

9. Interaction modes: direct, consult, debate, panel, handoff

Interaction modes are selected explicitly or inferred in auto mode. Direct mode uses one active agent. Consult mode asks supporting agents for private input before the primary agent responds. Panel mode collects multiple panelist responses and synthesizes them.

Debate mode creates a proposal, asks supporting agents to challenge it, and synthesizes a response. Handoff mode changes the primary agent to a supporting specialist when the request belongs elsewhere.

- direct: one agent answers.
- consult: active agent consults selected specialists.
- panel: multiple agents provide perspectives.
- debate: proposal plus critique workflow.
- handoff: transfer to the more relevant agent.

10. Supervisor / Director agent

The current code uses Supervisor plus `IntentRouter` as the director-like control layer. Supervisor detects module, vagueness, safety status, stuck signals, and completed deliverables. `IntentRouter` detects intent and selects primary/supporting agents.

The boss agent is also available as a user-selectable coordinator persona. It consults CEO, CHRO, and Regional Manager when broad synthesis is needed.

- `Supervisor.analyze` returns status, module, redirect flags, and recommended action.
- `IntentRouter` maps intent to primary and supporting agents.
- Boss agent provides a human-facing coordination persona.

11. Safety guardrails

`SafetyChecker` identifies prompt extraction, instruction override attempts, confidential data requests, final-answer outsourcing, off-track prompts, and harmful HR requests. The orchestrator blocks prompt-extraction attempts before prompt construction proceeds.

The UI no longer exposes a system-prompt preview. Public chat responses return retrieved-

context metadata only, reducing the chance of leaking internal configuration or prompt content.

- Prompt extraction returns a blocked response.
- Agent rules prohibit revealing hidden instructions.
- The reportable API metadata avoids raw prompt or raw retrieved text exposure.

12. Prototype implementation

The implementation is intentionally lightweight and local. It uses FastAPI for the API layer, static HTML/CSS/JavaScript for the UI, YAML/Markdown for agent configuration, and Python classes for routing, memory, safety, retrieval, and orchestration.

MockLLMClient provides deterministic English responses for reliable grading. GeminiLLMClient is optional and loaded only when .env requests Gemini and a real key is present.

- Entry point: app.py.
- UI: static/index.html, static/app.js, static/styles.css.
- Tests: tests/test_agent.py.
- Run guide: RUNNING.md.

13. Demo / sample API response

A typical browser request posts to /chat/{agent_id}. For example, posting an education communication request to regional_comms_manager returns a structured campaign-planning response and metadata about state, route, safety flags, and collaborators.

```
POST /chat/regional_comms_manager
{
  "session_id": "demo-session",
  "message": "I need to build an education communication project."
}

Sample response excerpt:
{
  "agent_id": "regional_comms_manager",
  "assistant_message": "For an education communication project, build the campaign around an audience journey...",
  "collaboration": {"collaborators": ["gucci_group_chro"]},
  "safety_flags": {},
  "context": [{"document_id": "module_3_rollout_adoption", "score": 0.74}]
}
```

14. Evaluation metrics

Evaluation should cover both engineering quality and assistant behavior. The prototype includes tests for agent list behavior, boss consultation, consult mode, debate mode, prompt-extraction blocking, hidden prompt endpoint removal, metadata-only retrieved context, mock fallback, and retriever metadata.

Product-level evaluation should measure whether the assistant is helpful, scoped, context-aware, safe, and faithful to the simulation.

- Functional: endpoint success, routing accuracy, mode selection, session continuity.
- Retrieval: relevant context, agent-scope filtering, no hidden prompt leakage.
- Safety: prompt-extraction refusal, assignment-scaffolding behavior, no confidential data

fabrication.

- UX: agent switching clarity, response depth, English consistency, browser reliability.
- Learning value: asks useful questions, scaffolds deliverables, avoids writing the final assignment end-to-end.

15. Limitations & next steps

The prototype is intentionally local and simplified. Mock embeddings and MockVectorDB approximate the production architecture but do not provide semantic retrieval quality equivalent to hosted embeddings or FAISS. MockLLMClient is deterministic and useful for grading, but it is not a substitute for a high-quality model in real use.

Next steps should focus on replacing mock components, expanding tests, adding richer observability, and improving evaluation with realistic user transcripts.

- Replace mock embeddings with a production embedding model.
- Add persistent vector DB storage and indexing scripts.
- Add authentication and request logging for hosted use.
- Add structured evaluation datasets and rubric-based scoring.
- Improve frontend state handling and add loading/error states.
- Add model-cost controls, retry behavior, and streaming responses.

Closing assessment

The source code presents a coherent local prototype for a multi-agent AI co-worker engine. Its strongest qualities are the separation between configuration and runtime, the API-first structure, the multi-mode orchestration design, the explicit safety layer, and the ability to run without paid credentials. The main engineering limitations are mock retrieval quality, non-persistent vector storage, deterministic mock responses, and the absence of production deployment concerns such as authentication, monitoring, and hosted storage.